

Architecture

Table of Contents

Architecture Documentation

- Selected stack and justification
- Backend architecture
- Cache management architecture
- Python analytics microservice
- URI conventions
- OAuth/session strategy
- Database model
- AWS deployment
- Environment variables
- Clean Code and SOLID decisions

Architecture Documentation

Selected stack and justification

Node.js with Express is used for a clear academic backend that supports REST, middleware, validation, testing and modular OOP service classes. React + Vite is selected as the frontend framework for the next UI phase. PostgreSQL is the production database and Prisma ORM is the required data-access layer because it supports an explicit schema, generated client and AWS RDS deployment.

Backend architecture

Controllers only parse HTTP concerns and delegate to services. Services implement business rules. Repositories isolate persistence. Validators centralize request validation. Middleware handles sessions, authorization, cache control, CORS and errors.

Runtime layers:

- Controllers: `AuthController`, `CrudController`, `AttendanceController`, `ReportsController`, `AcademicController`.
- Services: `AuthService`, `AccessPolicy`, `AttendanceService`, `RulesService`, `AcademicService`, `AuditService`, `CacheService`.
- Repositories: in-memory repositories for tests/local demos and Prisma repositories for production when `DB_DRIVER=prisma`.
- Middleware: validation, session resolver, RBAC, private page guard, explicit cache control and error handling.

RBAC decides whether a role can enter a workflow. `AccessPolicy` then checks the specific resource scope: BranchDirector users are limited by `user_branch_access`, Teacher users are limited to their linked teacher profile/classes, and Student users are limited to their own academic records.

Cache management architecture

The Express application creates one in-process `CacheService` at startup and passes it into controllers/services through dependency injection. The cache stores safe repeated reads with TTLs and tag invalidation. Cache keys include the authenticated actor role and id/email whenever the response can differ by role or branch scope.

HTTP cache policies are centralized in `src/middleware/cacheControl.js`:

- Sensitive API and private page responses use `Cache-Control: no-store, no-cache, must-revalidate, private`.
- Public OAuth client configuration uses a one-hour public cache.
- Reference catalog and branch/report reads use private browser cache plus server memory cache where safe.
- Memory cache evidence is exposed through `X-Memory-Cache`, `X-Memory-Cache-Key` and `X-Memory-Cache-TTL`.

State-changing services invalidate tags such as `branches`, `students`, `attendance`, `evaluations` and `reports`. This keeps repeated director dashboards fast while preventing stale academic data from remaining after writes.

Python analytics microservice

An additional Python FastAPI API is included under `06Code/python-analytics-api` for analytical academic endpoints. It is not responsible for creating sessions or writing transactional attendance records. It reads from the same PostgreSQL database and validates the same JWT session token issued by the Node Auth API.

Runtime responsibility:

- Base path: `/api/analytics/v1`.

- Health endpoint: public service check with a 60 second cache policy.
- Protected endpoints: student attendance risk, scholarship readiness, branch performance and teacher workload with no-store cache policy.
- Session validation: JWT signature check plus server-side token hash lookup in the shared `sessions` table.
- Resource authorization: analytics responses are filtered by the same student, teacher and branch scope rules used by the Node API.

URI conventions

Node private APIs use `/api/v1`. The Python analytics API uses `/api/analytics/v1`. Both use plural nouns where applicable, JSON request/response bodies and consistent envelopes: `{ success, message, data }` or `{ success:false, message, data:null }`.

OAuth/session strategy

The React login page supports email/password and Google Sign-In. Email is the academy username and `/api/v1/auth/login` authenticates existing users through `users.password_hash`. Google Identity Services sends an ID token to `POST /api/v1/auth/google`; the backend verifies it in production and links `google_sub` only to an existing active user with the same verified email. Google never creates private users and rejects unverified or mismatched Google emails. Admin and GeneralDirector users can create academy accounts from the private dashboard or `POST /api/v1/users`; Student and Teacher accounts are created together with their linked academic profile, while BranchDirector accounts require assigned branches through `user_branch_access`. New director-created accounts always require a first password change through `must_change_password`; protected academic endpoints reject those sessions until `/api/v1/auth/change-password` succeeds. Only a session token hash is stored server-side. Logout revokes the server-side session. Private pages use `Cache-Control: no-store` and a session guard that redirects anonymous users to `/login.html?session=expired`.

Database model

PostgreSQL schema is normalized around users/roles/permissions, explicit user branch access, branches, students, teachers, dance styles, class groups, class sessions, student attendance, teacher attendance, absence justifications, scholarship evaluations, level promotion evaluations, enrollment requests, sessions and audit logs.

AWS deployment

- Frontend EC2: Nginx on ports 80/443 serving static landing/app assets.

- Core Business API EC2: port 3000 for branches, students, teachers, classes and attendance.
- Auth & Session API EC2: port 3001 for OAuth, sessions, roles and permissions.
- Reports & Rules API EC2: port 3002 for scholarship, promotion, hours and payments.
- Python Analytics API EC2: port 8000 for attendance risk, scholarship readiness, branch performance and workload summaries.
- PostgreSQL: Amazon RDS private subnet on port 5432. Security groups should allow public 443 only to ALB/Nginx, API traffic only from frontend/ALB, and RDS only from API security groups. HTTPS with ACM certificates and an optional ALB is recommended.

Environment variables

See `06Code/.env.example` for required runtime variables.

Clean Code and SOLID decisions

Business rules live in services, HTTP details live in controllers, cross-cutting concerns live in middleware, and constants avoid magic strings. Classes depend on abstractions supplied through constructors.